Mipmaps: A Simple Q&A

Q: What acts as a Mipmap?

A: Imagine a set of **Russian Nesting Dolls**, but for pictures. A "Mipmap" isn't just one image; it's a whole stack of images. You have the original big photo (Level 0), then a copy that's half that size (Level 1), then quarter size (Level 2), all the way down to a tiny 1x1 pixel dot.

Q: Why on earth do we need tiny versions of our textures?

A: Two big reasons:

1. **To stop the "shimmer":** Have you ever seen a fence or a tiled floor look "noisy" or sparkle when it's far away in a video game? That's because the computer is trying to squeeze a huge detailed image into a few tiny screen pixels, and it gets confused. Using a smaller (blurrier) mipmap makes it look smooth and clean.
2. **Speed:** It's much faster for the graphics card to read a tiny image for a tiny object than to read a giant 4K image just to color 3 pixels.

Q: How did *we* make them in this project?

A: We didn't make them in Photoshop! We managed to make the graphics card (GPU) build them for us automatically when the game starts. Inside `labut2/vkimage.cpp`, we wrote a loop that does this:

> "Take the Main Picture -> Shrink it by half -> Save it as Level 1." "Take Level 1 -> Shrink it by half -> Save it as Level 2." (We used a Vulkan command called `vkCmdBlitImage` to do the shrinking/copying).

Q: Effect: How do we know it's working?

A: In the normal game mode (Key 1), you just see nice smooth textures that don't sparkle in the distance. But if you press **Key 2**, we turn on "Mipmap Debug Mode".

- **RED** = You are close to the object (Using the biggest, most detailed image).
- **BLUE/GREEN** = You are far away (Using the smaller, faster images). This proves the computer is automatically switching between the big and small images depending on distance!